Cairo University

Faculty of Computers and Artificial Intelligence

CS251

Introduction to Software Engineering

Personal Budgeting

Software Design Specifications

Version 1.0

Team Names and Emails

CS251: <Team Name>
Project: <FLOOOOOS>

Software Design Specification

04-2026

Contents

Team 3

Document Purpose and Audience 3

System Models 4

I. Architecture Diagram 4

II. Class Diagram(s) 5

III. Class Descriptions 6

IV. Sequence diagrams 7

Class - Sequence Usage Table 18

V. State Diagram

VI. SOLID Principles

VII. Design Patterns

Tools 21

Ownership Report 21

CS251: <Team Name>
Project: <FLOOOOOS>

Software Design Specification

Team

ID	Name	Email	Mobile
20240485	Mohamed Tamer	20240485@stud.fci-cu.edu.eg	015 51132324
20240124	Beshoy Fomail	20240124@stud.fci-cu.edu.eg	01202042111
20240448	Mazen Magdy	20240448@stud.fci-cu.edu.eg	
20240366	Omar Hassan	20240366@stud.fci-cu.edu.eg	

Document Purpose and Audience

Purpose

This document is to identify and outline the Personal Budgeting Software's requirements according to the agreement of its stakeholders.

The goal of the Personal Budgeting Software is to enable users to manage their financial resources more efficiently through tracking of income, expenditures, savings and any financial objectives users may have.

Other features offered by the Personal Budgeting Software are budgeting, reports and insights to assist with decision-making related to financial resources.

To improve the time/cost of developing this product are a primary function of this Personal Budgeting Software product.

Audience

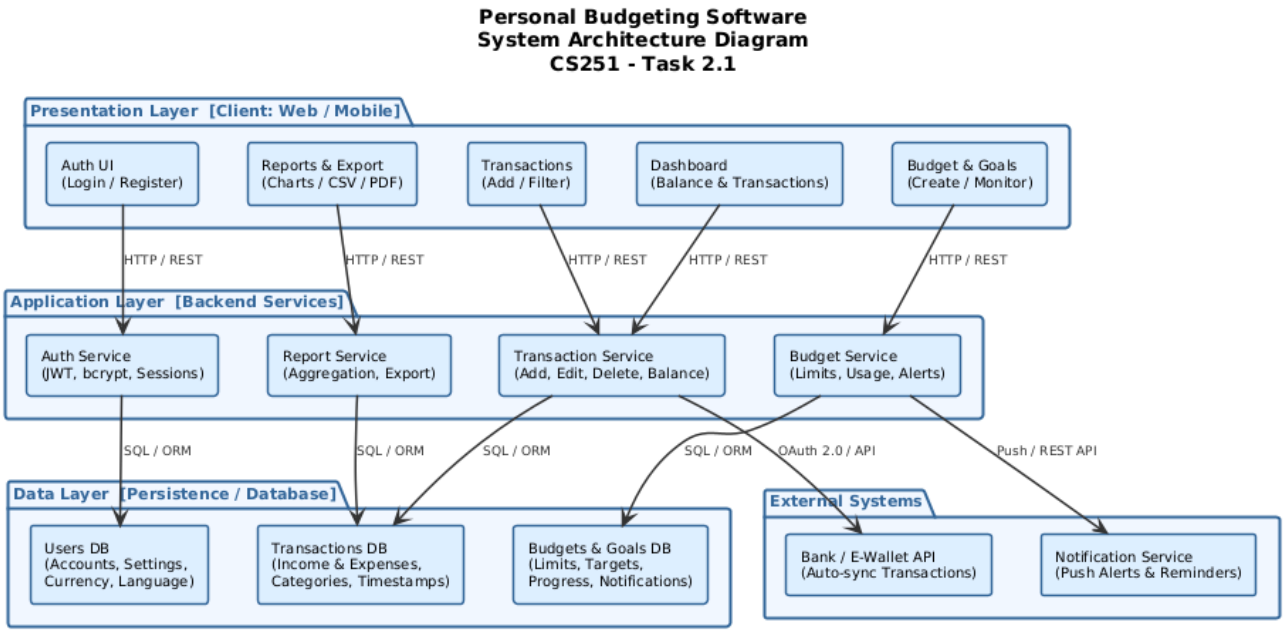
- **Software Development Team**
- **Stakeholders**
- **Project Manager**
- **End Users**

CS251: <Team Name>
Project: <FLOOOOOS>

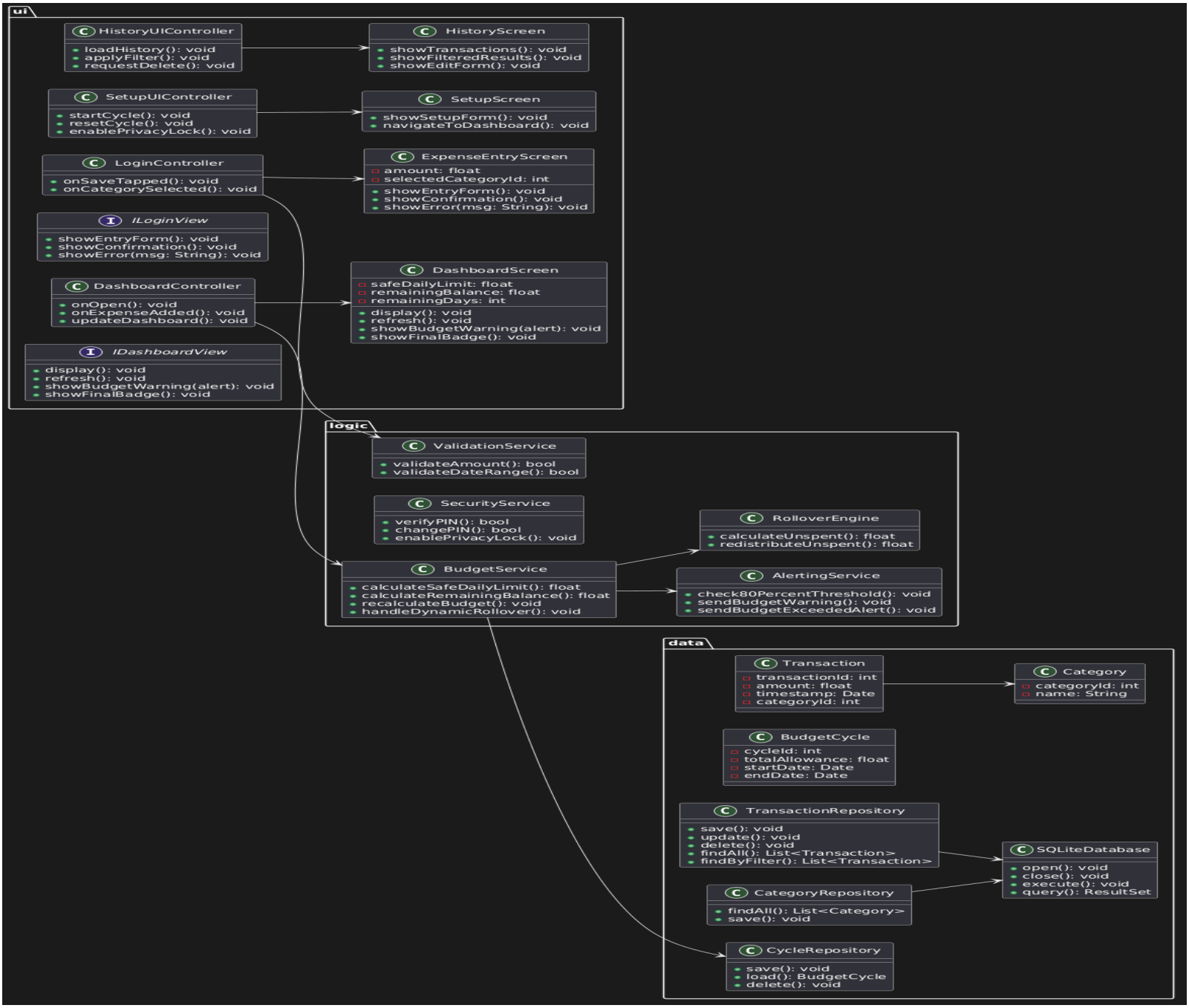
Software Design Specification

System Models

I. Architecture Diagram



Software Design Specification



III. Class Descriptions

Software Design Specification

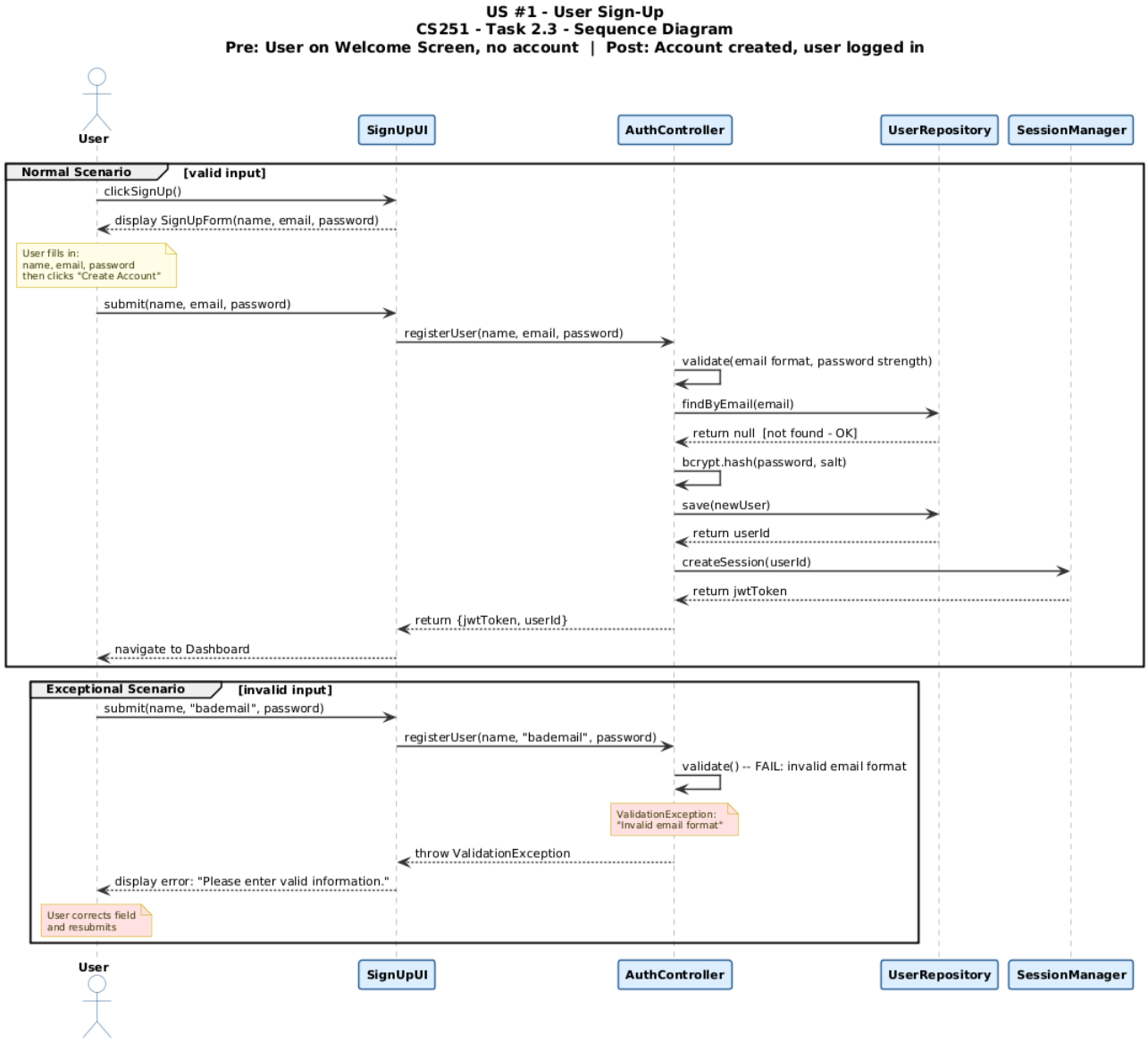
Class ID	Class Name	Description & Responsibility
1.	DashboardController	Responsible for handling dashboard-related actions such as loading data and updating the dashboard view.
2.	LoginController	Handles user login process, including validating input and communicating with authentication logic.
3.	SetupUIController	Manages the setup process of the application, including initializing budget cycles and enabling security features.
4.	HistoryUIController	Responsible for loading transaction history, filtering data, and handling edit/delete operations.
5.	DashboardScreen	Represents the main dashboard UI that displays financial summaries such as balance, daily limits, and alerts.
6.	ExpenseEntryScreen	Provides the interface for entering new transactions and validating user input.
7.	SetupScreen	Displays setup forms for initializing user preferences and budget settings.
8.	HistoryScreen	Displays transaction history and allows filtering and editing of transactions.
9.	ValidationService	Validates user input such as transaction amount and date ranges.
10.	SecurityService	Handles security-related features such as PIN verification and privacy lock.
11.	AlertingService	Responsible for generating alerts when budget thresholds are reached or exceeded.
12.	BudgetService	Handles all budget calculations including daily limits, remaining balance, and rollover logic.
13.	RolloverEngine	Manages leftover budget and redistributes unspent amounts across cycles.
14.	CycleRepository	Handles database operations related to budget cycles such as saving and loading data.
15.	TransactionRepository	Manages CRUD operations for transactions.
16.	CategoryRepository	Handles storage and retrieval of transaction categories.
17.	BudgetCycle	Represents a budgeting period including total allowance and date range.
18.	Transaction	Represents a financial transaction with amount, category, and timestamp.
19.	Category	Represents a transaction category such as food, transport, etc.

Software Design Specification

Class ID	Class Name	Description & Responsibility
20.	SQLiteDatabase	Handles low-level database operations such as executing queries and managing connections.

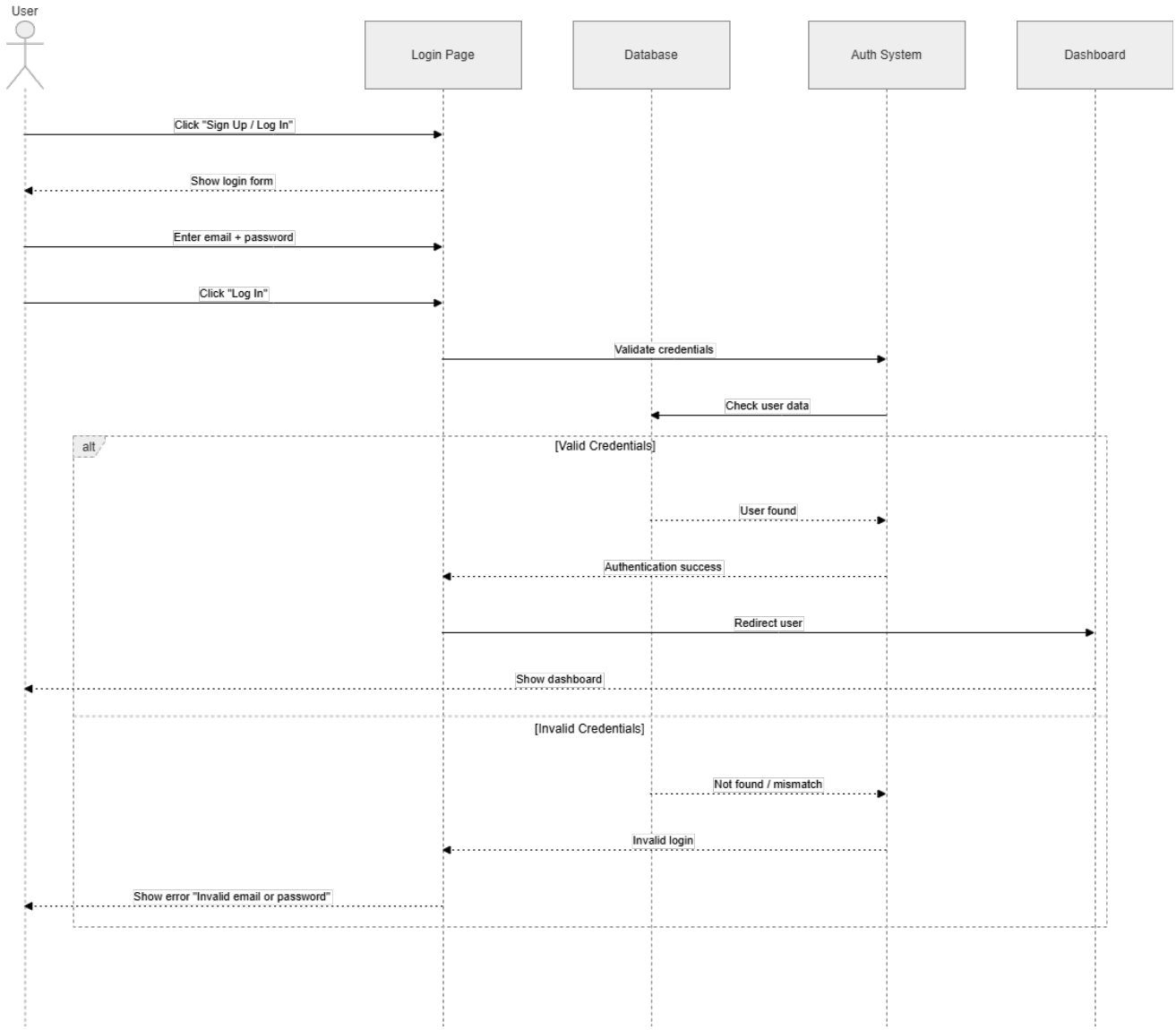
IV. Sequence diagrams

User story 1



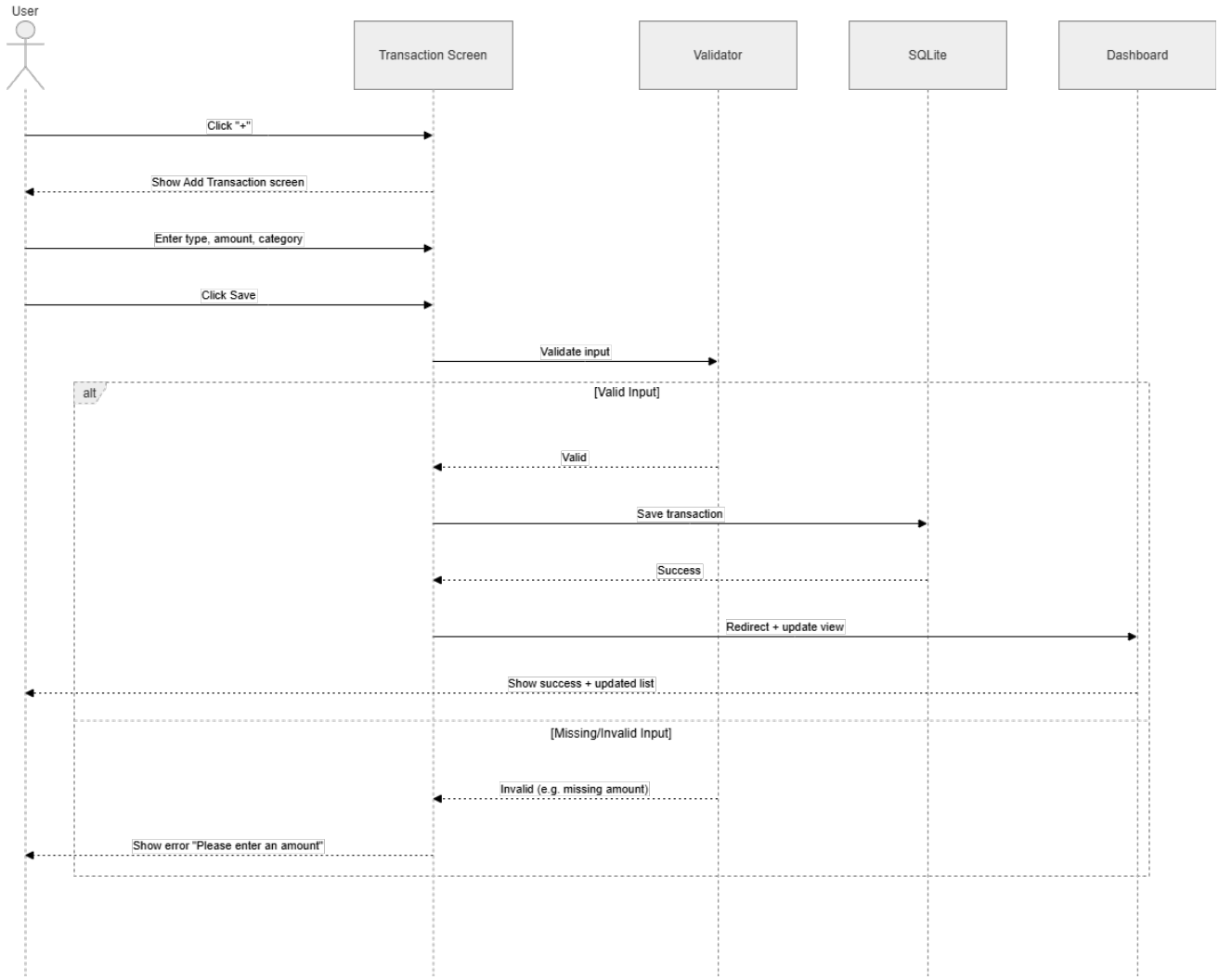
Software Design Specification

User story 2



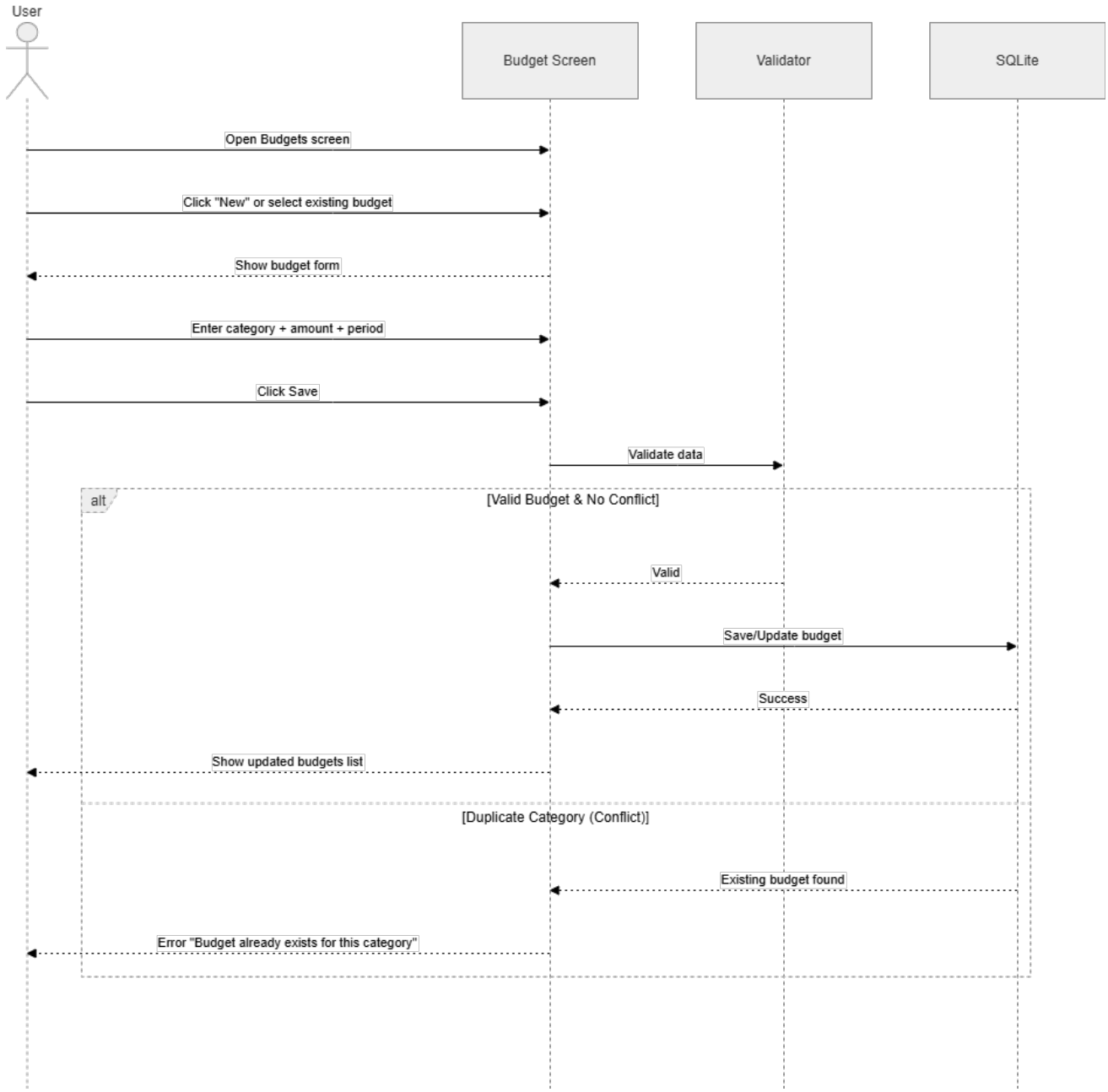
Software Design Specification

User story 3



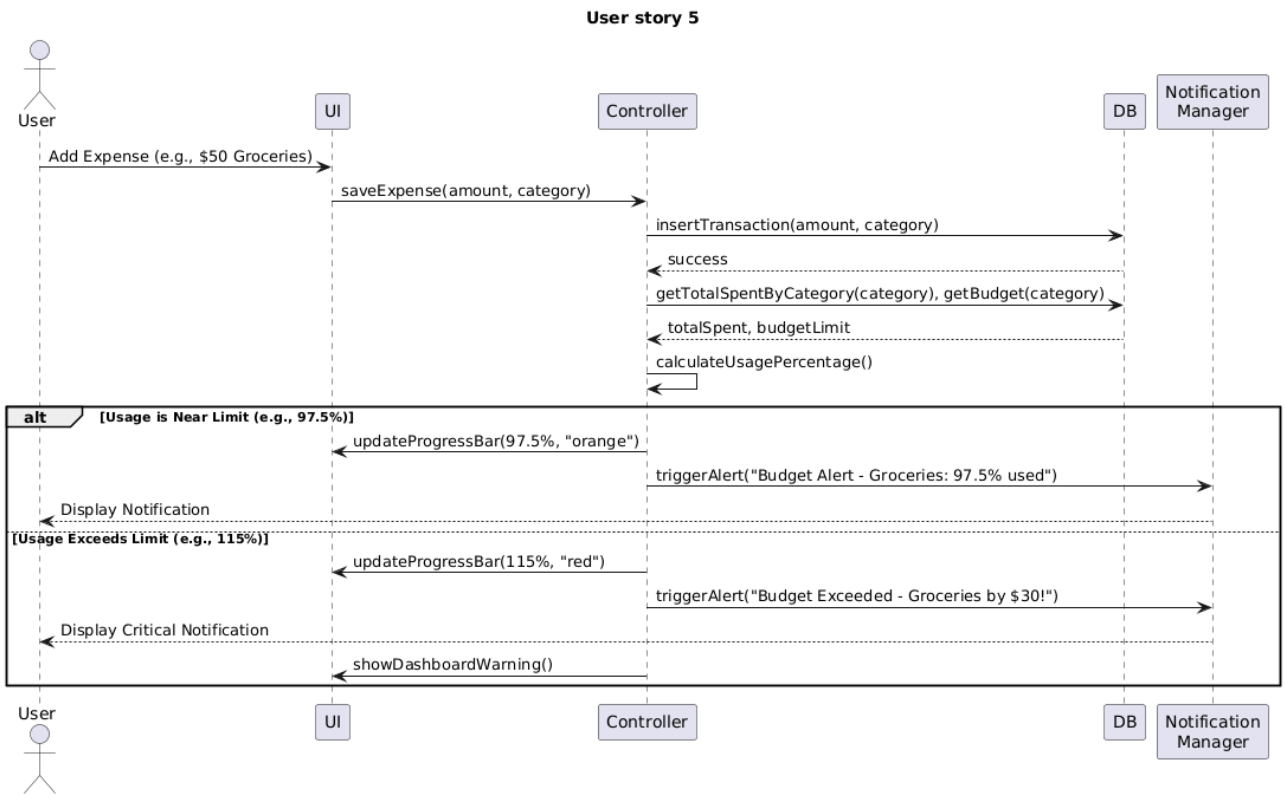
Software Design Specification

User story 4



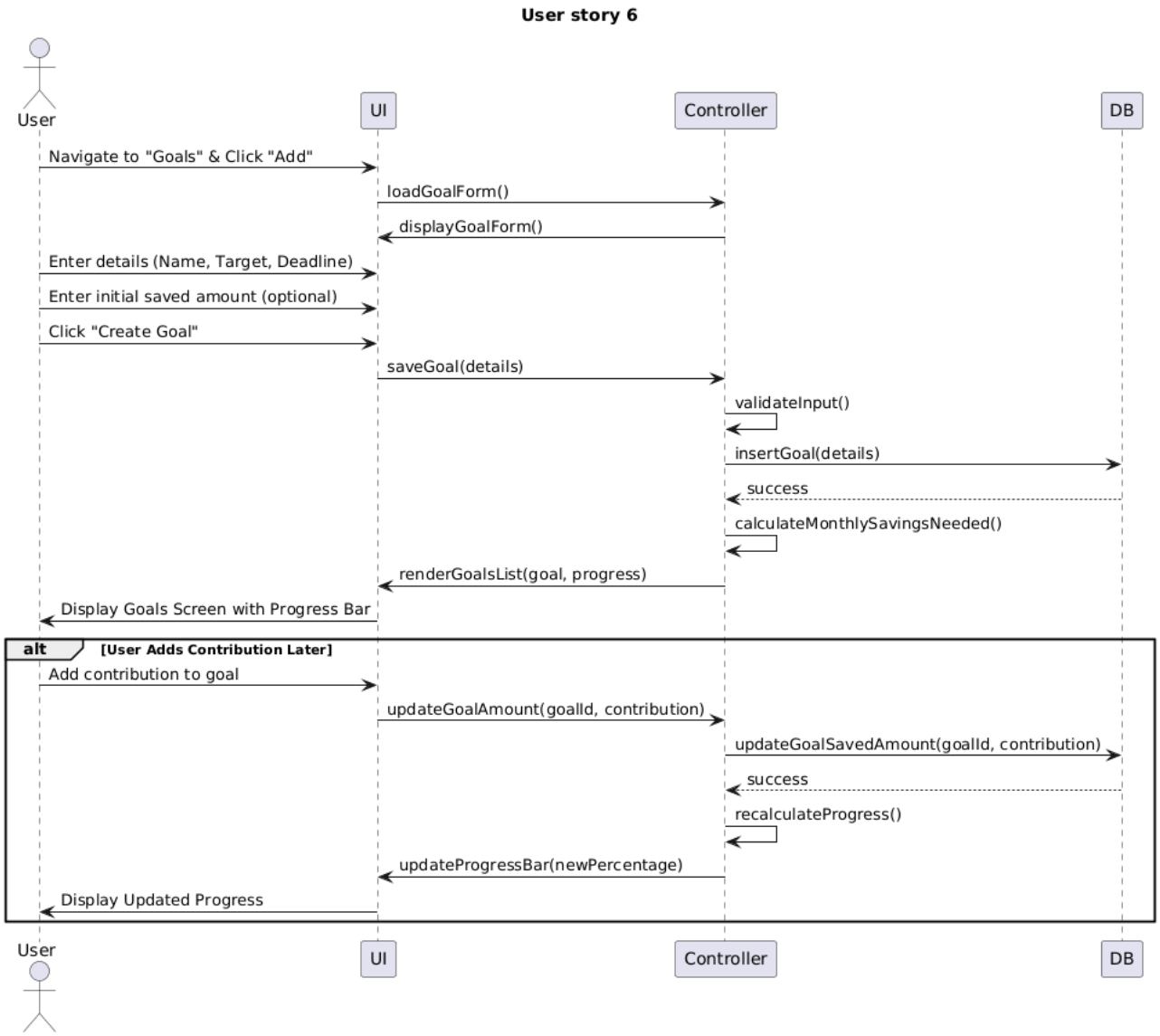
Software Design Specification

User story 5



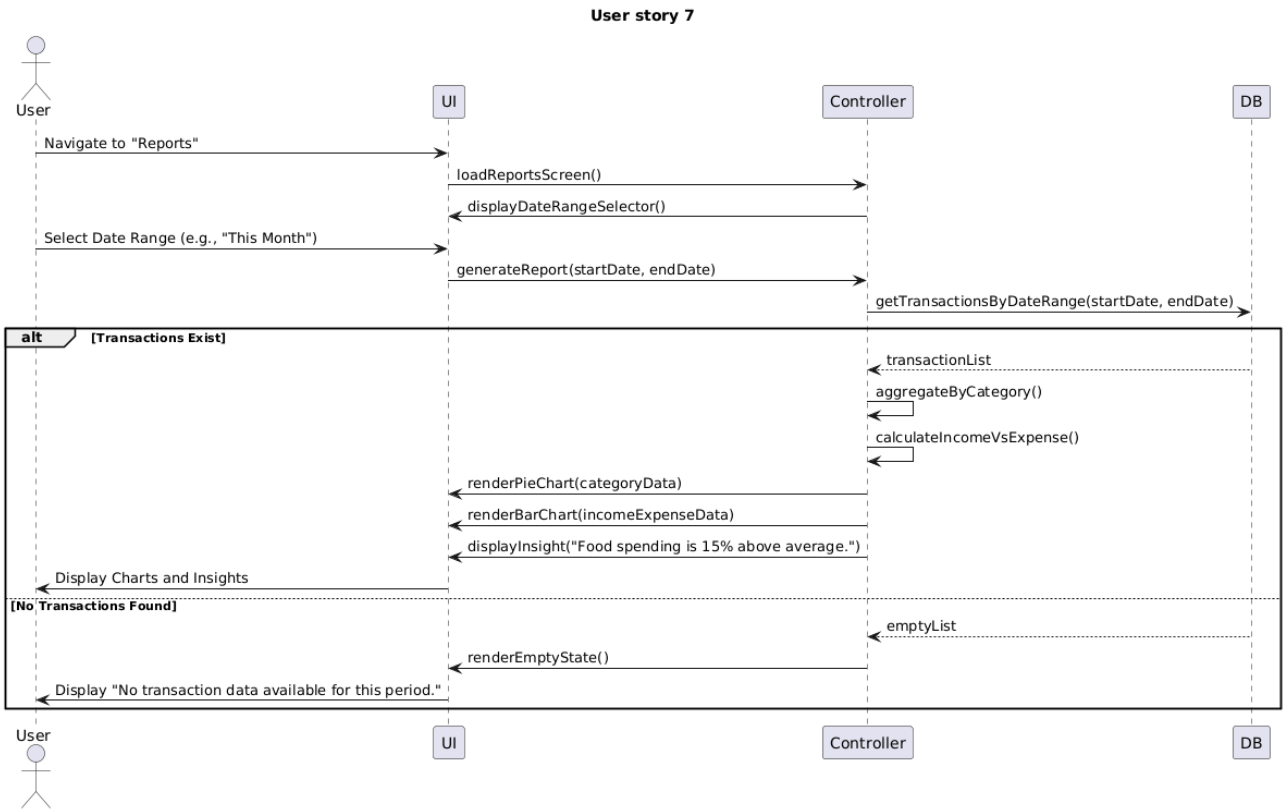
Software Design Specification

User story 6



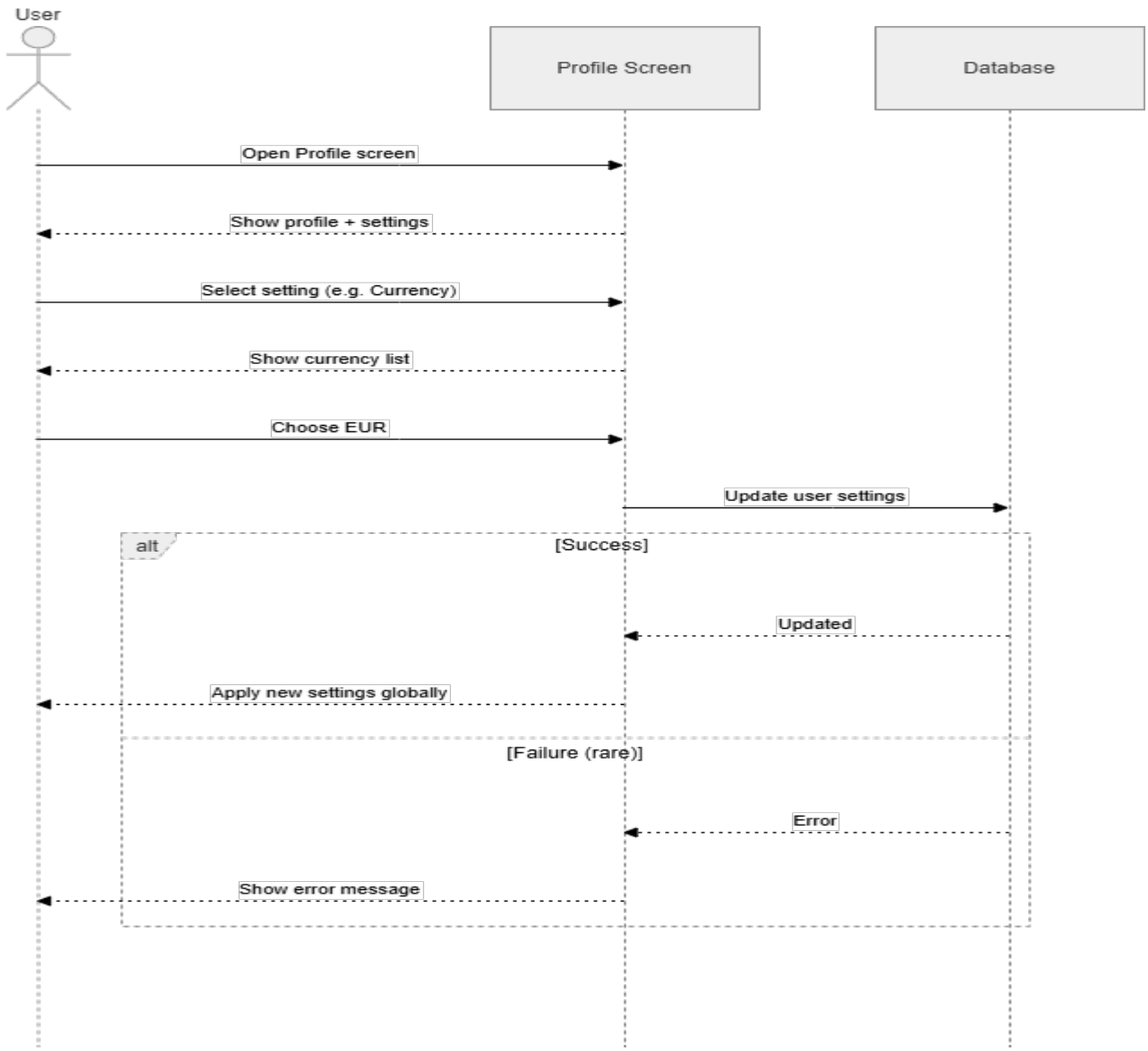
Software Design Specification

User story 7



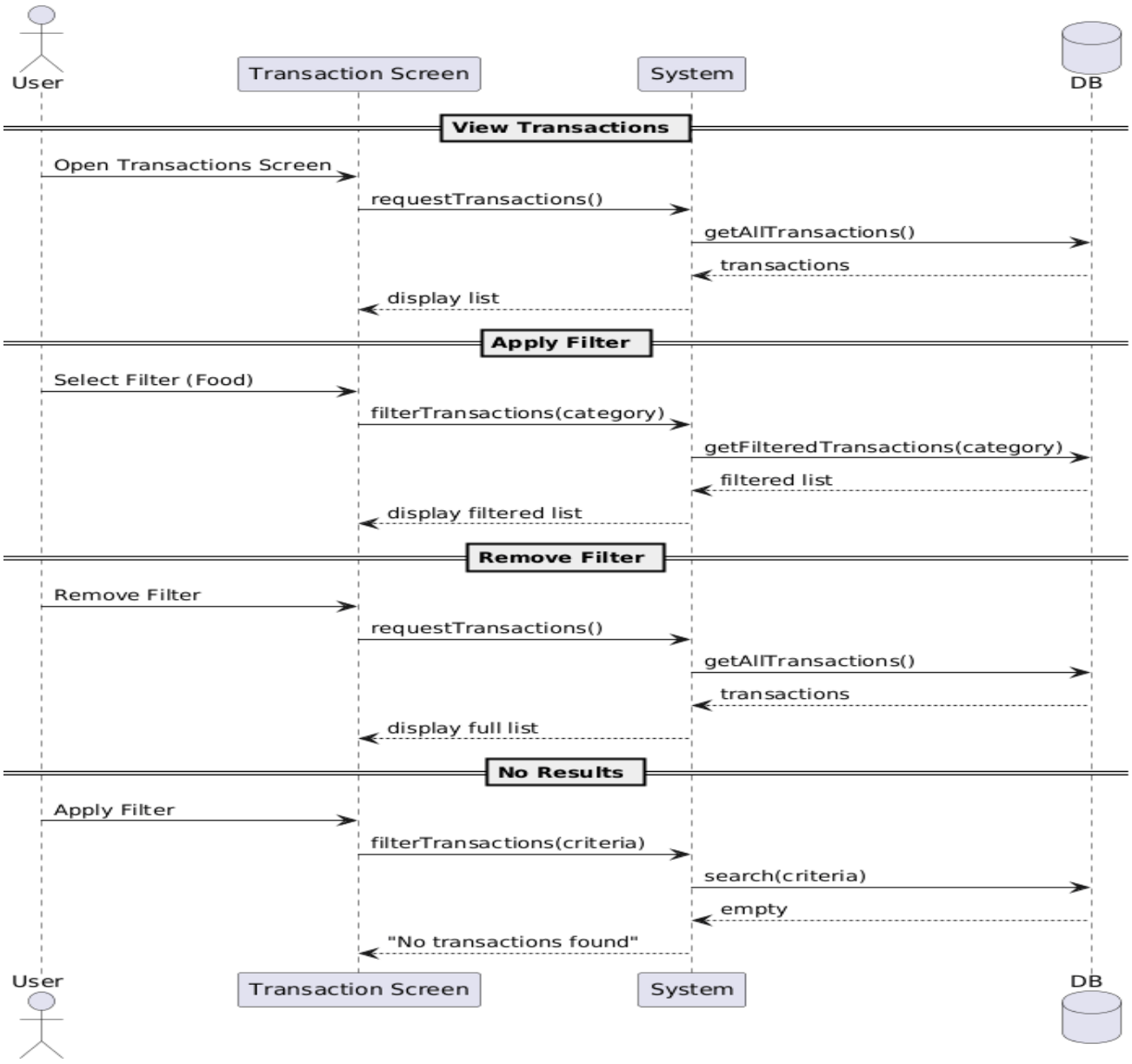
Software Design Specification

User story 8



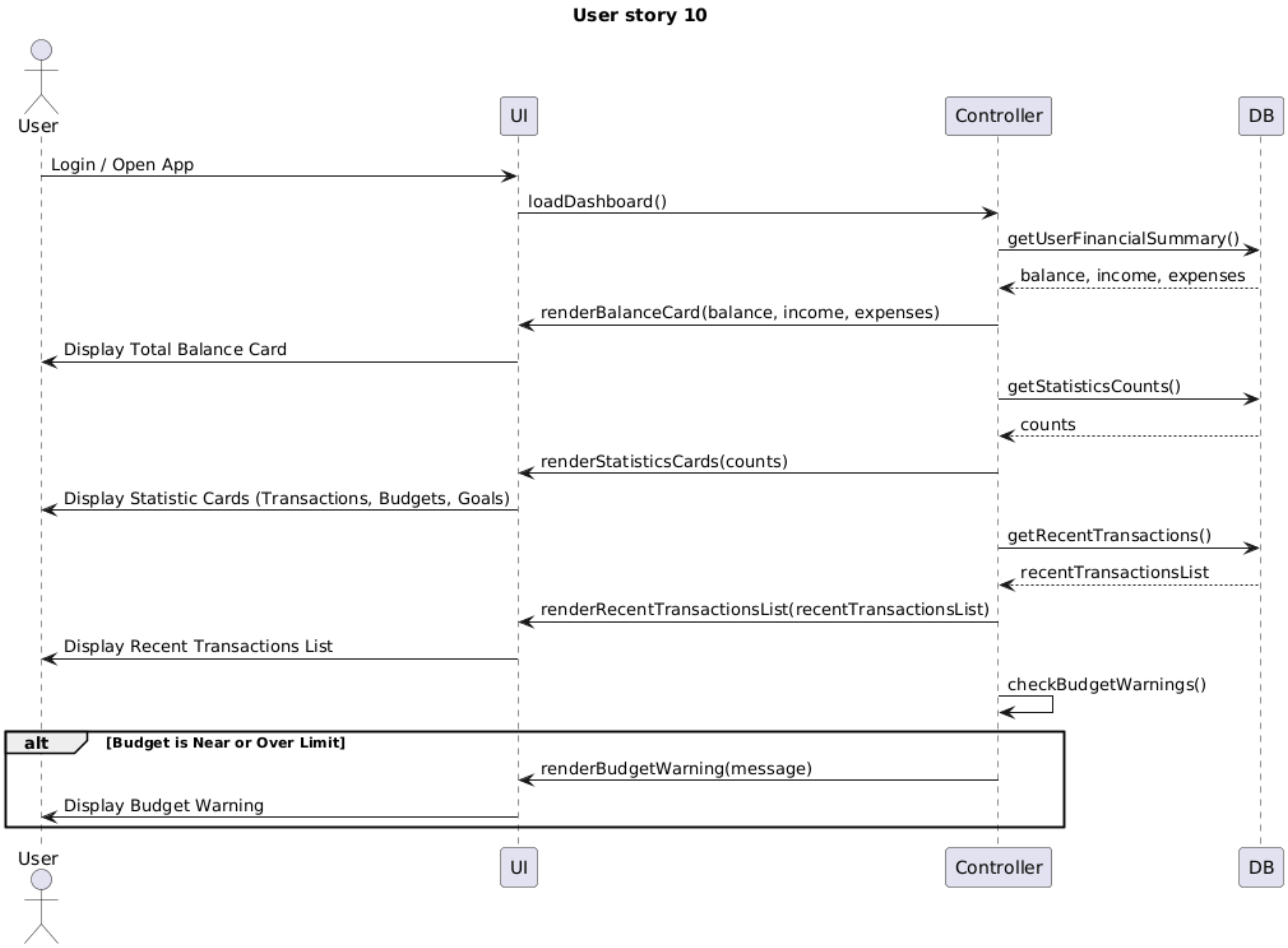
Software Design Specification

User story 9



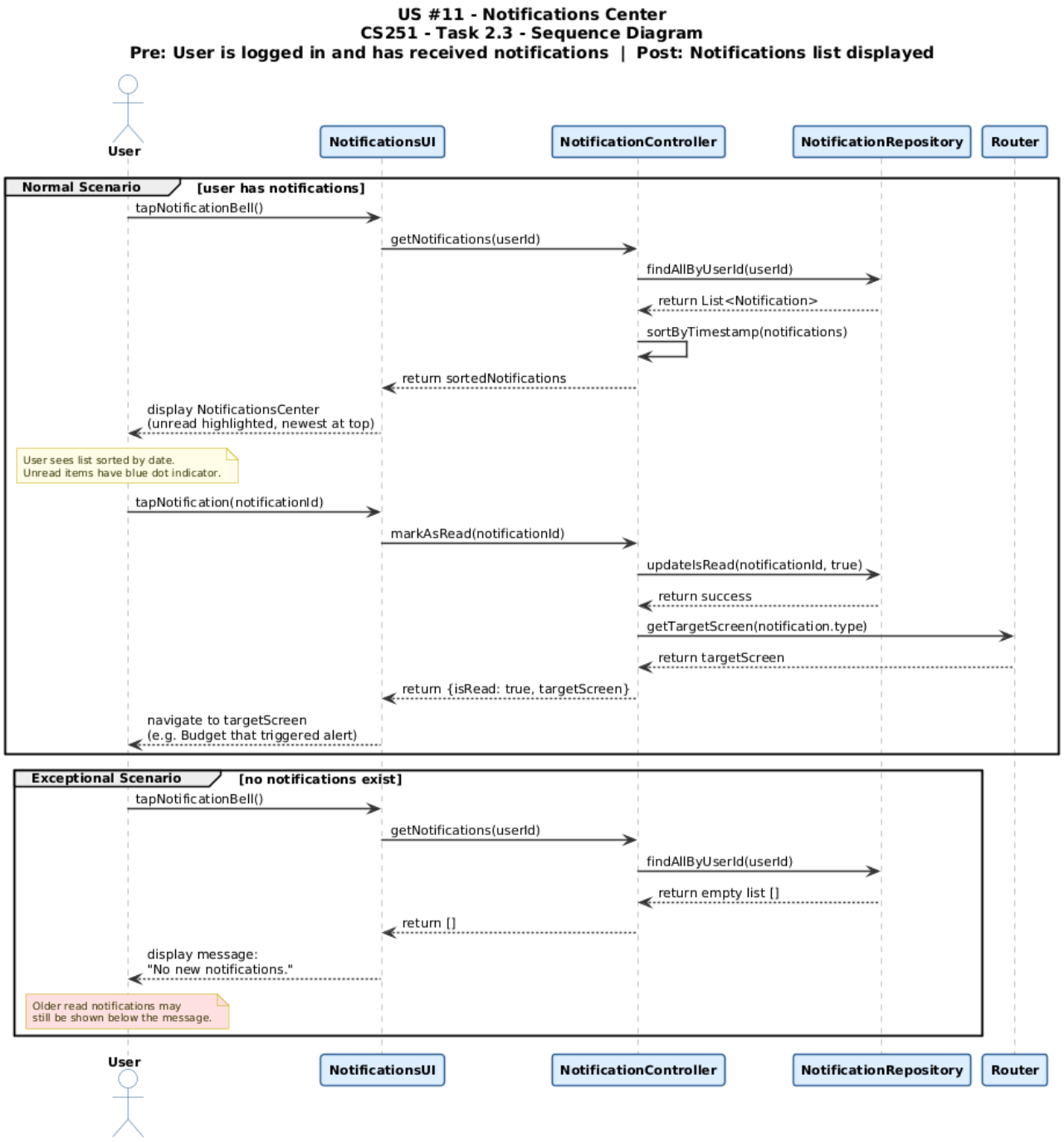
Software Design Specification

User story 10



Software Design Specification

User story 11



Software Design Specification

Class - Sequence Usage Table

Sequence Diagram	Classes Used	All Methods Used
1. User Sign-Up(US #1)	SignUpUI	clickSignUp() submit(name, email, password)
	AuthController	registerUser(name, email, password) validate(email, password) bcrypt.hash(password, salt)
	UserRepository	findByEmail(email) : User save(newUser) : userId
	SessionManager	createSession(userId) : jwtToken
2. Log In(US #2)	LoginController	onSaveTapped() onCategorySelected()
	ILoginView	showEntryForm() showConfirmation() showError(msg: String)
	ValidationService	validateAmount() : bool validateDateRange() : bool
	SQLiteDatabase	open() close() query() : ResultSet
3. Add Transaction(US #3)	ExpenseEntryScreen	showEntryForm() showConfirmation() showError(msg: String)
	ValidationService	validateAmount() : bool
	TransactionRepository	save() : void
	SQLiteDatabase	execute() : void query() : ResultSet
4. Create/EditBudget(US #4)	SetupUIController	startCycle() resetCycle() enablePrivacyLock()
	SetupScreen	showSetupForm() navigateToDashboard()
	BudgetService	calculateSafeDailyLimit() : float calculateRemainingBalance() : float recalculateBudget()

Software Design Specification

Sequence Diagram	Classes Used	All Methods Used
		handleDynamicRollover()
	CycleRepository	save() : void load() : BudgetCycle delete() : void
5. BudgetOver-Limit Alert(US #5)	AlertingService	check80PercentThreshold() : void sendBudgetWarning() : void sendBudgetExceededAlert() : void
	BudgetService	calculateSafeDailyLimit() : float calculateRemainingBalance() : float handleDynamicRollover() : void
	TransactionRepository	save() : void
	SQLiteDatabase	execute() : void
6. Set & TrackFinancial Goals(US #6)	DashboardController	onOpen() onExpenseAdded() updateDashboard()
	DashboardScreen	display() refresh() showBudgetWarning(alert) showFinalBadge()
	BudgetService	calculateSafeDailyLimit() : float calculateRemainingBalance() : float
	TransactionRepository	save() : void findAll() : List<Transaction>
7. GenerateFinancial Reports(US #7)	DashboardController	onOpen() updateDashboard()
	DashboardScreen	display() refresh()
	TransactionRepository	findAll() : List<Transaction> findByFilter() : List<Transaction>
	RolloverEngine	calculateUnspent() : float redistributeUnspent() : float
8. Edit Profile& Settings(US #8)	HistoryUIController	loadHistory() : void applyFilter() : void requestDelete() : void

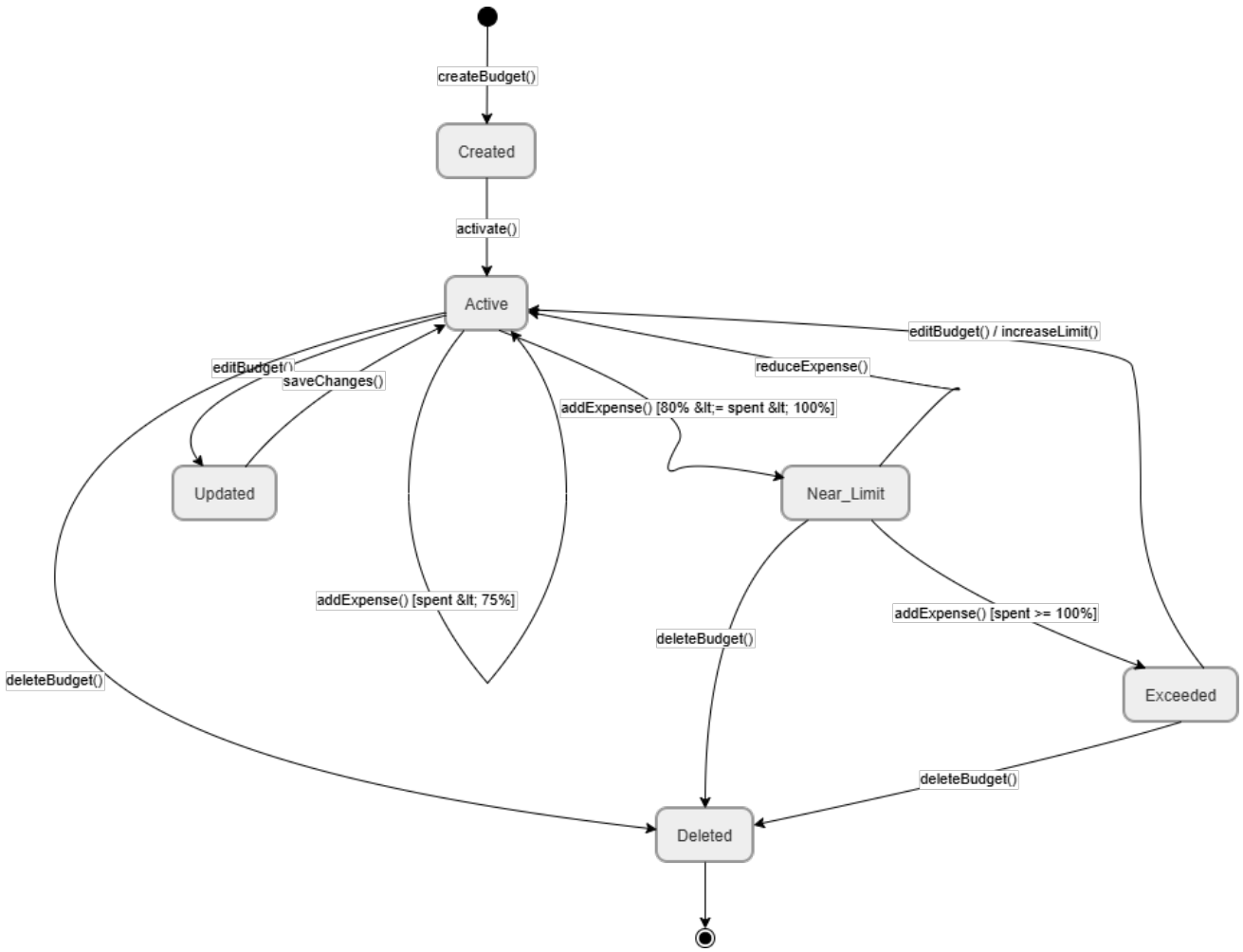
Software Design Specification

Sequence Diagram	Classes Used	All Methods Used
	HistoryScreen	showTransactions() : void showFilteredResults() : void showEditForm() : void
	ValidationService	validateAmount() : bool
	TransactionRepository	findAll() : List<Transaction> update() : void delete() : void
	SQLiteDatabase	execute() : void query() : ResultSet
9. Filter & ViewTransactionHistory(US #9)	HistoryUIController	loadHistory() : void applyFilter() : void
	HistoryScreen	showTransactions() : void showFilteredResults() : void
	TransactionRepository	findAll() : List<Transaction> findByFilter() : List<Transaction>
	SQLiteDatabase	query() : ResultSet
10. DashboardOverview(US #10)	DashboardController	onOpen() onExpenseAdded() updateDashboard()
	IDashboardView	display() refresh() showBudgetWarning(alert) showFinalBadge()
	BudgetService	calculateSafeDailyLimit() : float calculateRemainingBalance() : float
	CycleRepository	load() : BudgetCycle
	TransactionRepository	findAll() : List<Transaction>
11. NotificationsCenter(US #11)	NotificationsUI	tapNotificationBell() tapNotification(notificationId)
	NotificationController	getNotifications(userId) : List<Notification> sortByTimestamp(notifications) markAsRead(notificationId) getTargetScreen(type) : Screen
	NotificationRepository	findAllByUserId(userId) : List<Notification>

Software Design Specification

Sequence Diagram	Classes Used	All Methods Used
		updateIsRead(notificationId, isRead) : void
	Router	getTargetScreen(notificationType) : Screen

V. State Diagram



Software Design Specification

VI. SOLID Principles

SOLID Principles Application

1. Single Responsibility Principle (SRP)

A single responsibility is assigned to each of the classes in the application.

E.g.:

- HistoryUIController class is responsible for managing transactions' history only (filtering and loading).

- TransactionRepository is responsible for the data operations (methods findAll and findByFilter).

- BudgetService is responsible for budget processing.

Each class in the system depends on a certain responsibility making them easier to change.

2. Dependency Inversion Principle (DIP)

Instead of being implemented using low-level modules, high-level modules depend on abstractions.

E.g.:

- High-level modules like controllers (HistoryUIController) depend on services and repositories, but not on the lower-level modules, such as the database itself.

- TransactionsRepository class abstracts from the data operations, while SQLiteDatabase class implements them.

Dependency on abstractions makes the system easier to modify and to expand, for example, replacing the data source is possible without modifying higher-level modules.

3. Interface Segregation Principle (ISP)

An interface must provide the method that is needed for its users.

E.g.:

- Each screen in the UI module performs its functions differently (HistoryScreen, DashboardScreen, ExpenseEntryScreen). Therefore, each of these screens has a set of unique methods.

- There is a number of different services (ValidationService, SecurityService, AlertingService), implementing various functionality.

VII. Design Patterns

Strategy Pattern :

Design Pattern for Strategy Pattern Usage

The use of the Strategy Pattern in the described case will be in order due to the ability to select different strategies at runtime.

In this context, the Strategy pattern is utilized in the budget calculation and rollover operations.

- BudgetService serves as the Context.

- RolloverEngine class is the concrete strategy performing rollover operation.

By using this approach, we avoid hardcoding our budget calculation mechanism and delegate this responsibility to some other component which can vary from time to time.

There may be several strategies available, including:

- StandardRolloverStrategy (roll over remaining budget into the next cycle);

- NoRolloverStrategy (budget reset at the beginning of each new cycle);

- SmartRolloverStrategy (redistributing budget according to categories).

Advantages of this solution include:

- Improved flexibility and extensibility.

- The principle of open/closed is supported.

- System maintenance becomes easier.

CS251: <Team Name>
Project: <FLOOOOOS>

Software Design Specification

Tools

- **Draw.io , Visual paradigm**

Ownership Report

Owners	Item
Mohamed Tamer	Part of class diagram and sequence diagrams 9 , 3 of SOLID principles , Strategy Pattern
Beshoy Fomail	Architecture Diagram & sequence diagram 1 , 11
Mazen Magdy	sequence diagrams 5 , 6 , 7 , 10
Omar Hassan	sequence diagrams 2 , 3 , 4 , 8